Language Identification for Sinhala Script

A Benchmark Dataset and Model Suite for Script-Ambiguous Text

TEAM MEMBERS

Kularathna D.V.N.

Kulathunge K.A.N.H.

Kumarasinghe M.P.

PROJECT GUIDANCE

Mentor
Dr. Nisansa de Silva

Teaching Assistant
B.H.Dilanka Heshan

Trilingual Benchmark Dataset & Model Suite

This project develops the first language identification benchmark to distinguish between Sinhala, Pali, and Sanskrit written in the Sinhala script.

Goal

Correct the flaw where systems over classify all Sinhala script text as Sinhala.

Approach

Construct a high-quality trilingual corpus from SiPaKosa, SiDiaC-v.2.0, and open digital libraries.

Methodology

Evaluate sub-word and sequential feature models, spanning traditional ML (SVMs, CRFs), deep learning, and fine-tuned multilingual transformers.

Deliverables

Open-source benchmark dataset, rigorous comparative evaluation, deployment-ready classifiers, and a peer-reviewed research paper.

The Myth of 'Script Equals Language'

Current LangID systems assume script equals language, treating all Sinhala-script text as Sinhala. This misses historically significant Pali and Sanskrit texts.

01

The Flaw

No public dataset or model can make this three-way distinction.

02

The Challenge

These languages share a script and vocabulary, frequently code-mixing within a single paragraph.

03

Impact

Archives struggle to separate texts, OCR and translation models produce silent errors, and search indexes degrade for scholars.

Digital Humanities & Translation Flow

The inability to distinguish these languages has practical, widespread consequences:

Digital Humanities & Archives

Sri Lankan temple libraries and national archives cannot automatically separate Pali suttas from Sinhala commentary in scanned books.

Machine Translation & OCR

Feeding Pali text into a Sinhala translation or spell-checking model produces garbage output, and errors propagate silently.

Religious & Scholarly Communities

Monks and scholars searching digitized Tripitaka commentaries get poor retrieval results because search indexes mix the three languages.

Assembling 900K+ Script-Ambiguous Sentences

We are constructing a sentence-level dataset of roughly 900,000+ sentences by combining publicly available corpora under a unified schema.

01 . Schema

Data Structure

Every sentence is stored under a unified structure:

id, text, label, source, subcorpus, group_id

02 . Strategy

Splitting Strategy

The **group_id** enables grouped splitting, ensuring sentences from the same book never appear in both train and test sets.

Prevents data leakage.

03 . Validation

Unified Split

This creates a leak-free **train / validation / test** split.

Ensures highly reliable and robust model evaluation metrics across different domains.

SiDiaC-v2.0 and SiPaKosa

Our primary sources for Sinhala and Pali data include:

SiPaKosa

Provides ~148K Pali + Sinhala rows. It is an open HuggingFace dataset (RaniduG/SiPaKosa-Sent) containing Sinhala, Pali, and Mixed labels.

SiDiaC-v2.0

A large book-level corpus of Sinhala text. It consists of OCR text files with <eos> markers, with labels inferred at the book level (GitHub: NeviduJ/SiDiaC-v.2.0).

Note: The 'mixed' label (code-switched sentences) from SiPaKosa is excluded from the core 3-way task and kept for separate analysis.

SansinNT and Digital Corpus of Sanskrit

MINORITY CLASS STATUS: Sanskrit represents only **~1% of the corpus**, requiring specialized data mapping pipelines.

SansinNT

eBible.org • CC BY-SA 4.0

~8K Verses

Minority Dataset

Format: USFX XML structure

Extraction: Verse text extracted from element tails

Digital Corpus of Sanskrit

DCS • CC BY 3.0

>600K

Annotated Sentences

Format: XML / CoNLL-U format

Transliteration: via Aksharamuka Lib

Traditional ML Character n-Grams

We evaluate traditional machine learning models as baselines:

01 **Models:** Logistic Regression, Naive Bayes, and SVM.

02 **Features:** Character n-gram features (sequences of 1–5 characters).

03 **Intuition:** Pali and Sanskrit use characteristic letter combinations and word endings that differ from Sinhala; counting them is highly effective for morphological pattern identification.

State-of-Art Models We Experimented

We evaluate and compare several state-of-the-art language identification models under a unified benchmarking framework.

01 . STANDARD

Global LID

fastText LID-176
Standard baseline for rapid, lightweight language identification.

NLLB LID-218
Massively multilingual model supporting low-resource scripts.

02 . ADVANCED

Specialized LID

GlotLID v3
Focused on massive, clean, and representative language coverage.

OpenLID-v2
An open, high-quality community-driven language identifier.

03 . FINE-TUNED

Neural & Custom

XLM-R Base
State-of-the-art representation model fine-tuned for classification.

ConLID
Our custom neural model optimized for local script identification.

Benchmarking Datasets
CommonLID, Flores Plus, Wili 2018

Comparison
Tables:https://docs.google.com/spreadsheets/d/18q-d5mDKl3FiGwRVONodQ7cB8SF0Y7MiUWvOhAazUgE/edit?usp=sharing

Macro-F1 Metric & Robustness Analysis

Our evaluation suite is designed to handle class imbalance and domain shifts:

Primary Metric

Macro-averaged F1 Scores

This ensures the minority Sanskrit class counts equally with the larger classes.

Detailed Evaluation

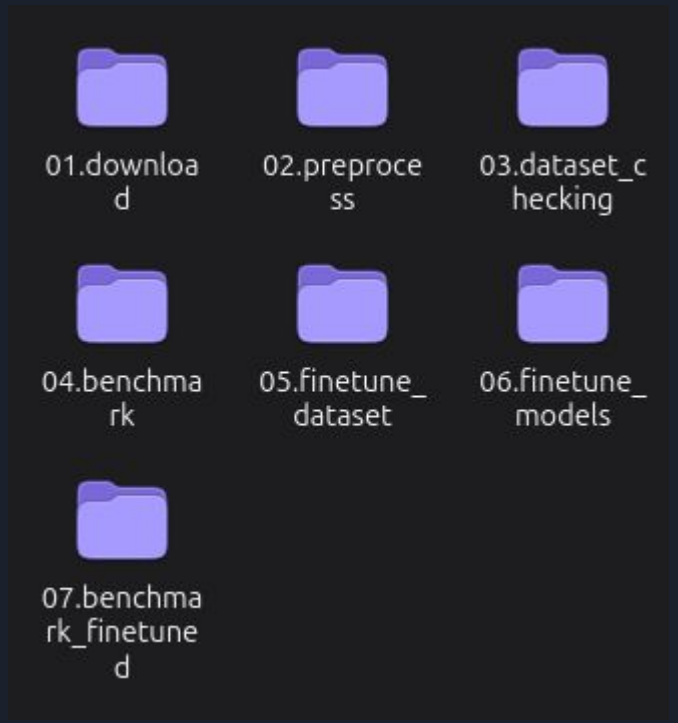
Detailed Metrics

Per-class precision, recall, F1, and confusion matrices to identify misclassified language pairs.

Data Pipeline Design

Modular scripts

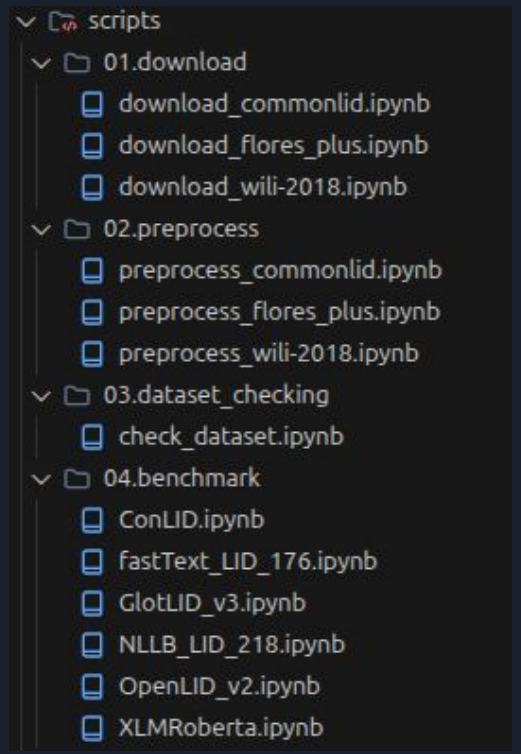
- **Standardized format mapping:** Disparate sources (CSV, XML, JSON, TXT), convert all data into uniform JSON Lines (.jsonl) schema (text, label, source).
- Downstream benchmarking and finetuning scripts only interact with standardized .jsonl.
- Each step processes unique actions giving ability to run each step of the pipeline anytime.

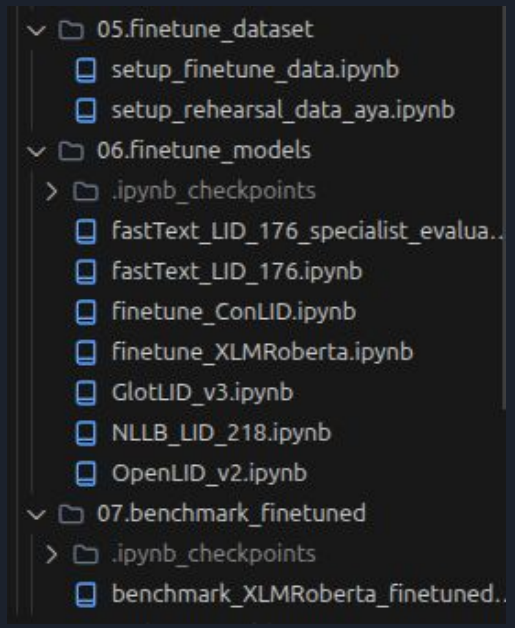


Pipeline In Depth

Our data pipeline ensures rigorous, reproducible dataset creation:

- The pipeline begins with a systematic approach to acquiring raw corpus data, followed by dedicated preprocessing modules designed to clean, normalize, and format the text
- Before any modeling occurs, a dedicated quality control phase validates data integrity, distribution, and cleanliness..
- The architecture establishes a clear baseline by evaluating out-of-the-box, pre-trained models on our datasets
- A distinct stage is used to carefully curate datasets specifically optimized for the fine-tuning process.





Production Architecture

API Gateway, Backend, and Next.js Frontend

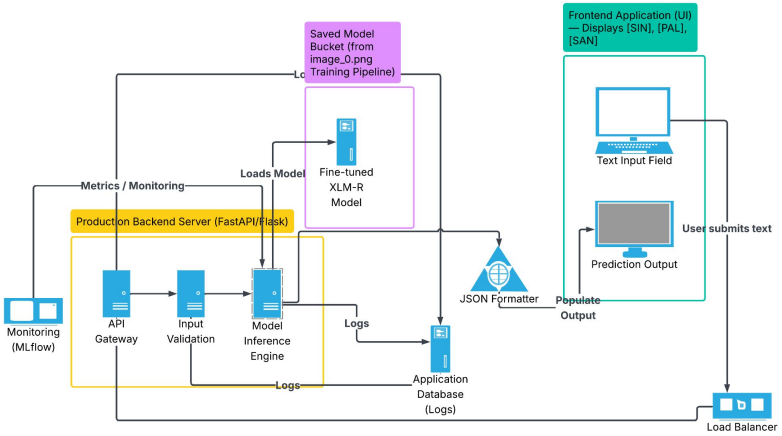
The project culminates in a live web application for real-time predictions:

Frontend: A Next.js application providing a live inference UI, confidence display, and color-coded linguistic markup.

Backend: A high-performance, containerized REST API (FastAPI) serving the models.

Stack: Python, HuggingFace, scikit-learn, fastText, FastAPI, NextJS, and Git/GitHub.

System Pipeline & Data Flow



Integrated Prediction Ecosystem

The production-ready design maps the live architecture from the **FastAPI Backend Server**, through the **Saved Model Bucket**, up to the interactive **Streamlit Frontend** UI.

Diagram highlights API Gateway, Input Validation, Inference Engine, and load balancing mechanics.

Individual Responsibilities

The project workload is divided among the three team members:

Kulathunge K.A.N.H.

- Baseline classifiers (Character n-grams + TF-IDF + Logistic Regression)
- Zero shot testing for NLLB and GlotLID
- Fine Tuning ConLID, GlotLID and NLLB
- User authentication interfaces - Login and Sign-up pages

Kumarasinghe M.P.

- Character-level neural networks
- Zero shot testing for FastText and OpenLID
- Fine Tuning FastText and OpenLID
- User dashboard implementation

Kularathna D.V.N.

- Transformer fine-tuning (XLM-R)
- Data pipeline engineering
- Backend Implementation of the webapp
- Testing OCR model

Expected Outcomes & Success Criteria for Public Release

Outcomes

Our project aims to deliver concrete, public-ready assets:

- A public dataset
- A comprehensive benchmark report
- Open-source trained models
- A live Streamlit web app
- An academic research paper

Success Criteria

- **Data integrity:** ≥3 sources, zero leakage
- **Thorough evaluation:** ≥4 model families
- **High performance:** Macro-F1 ≥ 0.95, identifying the rare Sanskrit class
- **Public readiness:** Paper drafted, dataset/models published

Future Integrations & Evaluation Milestones

Key initiatives focused on user-driven model choice and advanced Sinhala-Pali code-switched benchmarking:

Model Selection

Empower users with dynamic platform access to choose, configure, and swap active prediction model architectures on demand.

Code-Mix Analysis

Advance evaluation protocols for held-out bilingual datasets, specifically targeting complex mixed Sinhala–Pali text structures.

Benchmark Goal

Optimize target pipelines to ensure fine-tuned models consistently beat the baseline character n-gram baseline on macro-F1 scores.